

Take-Home: A Runtime for a Mixed Fleet of Arms

In one sentence: build the software that runs manipulation work across a fleet of factory cells that were not built the same, and keeps the line moving when individual cells can't do the work in front of them.

Time: ~5 hours. You will not finish everything; scoping down deliberately is part of the exercise. AI tools are allowed and encouraged.

What this is

You are simulating a factory on your laptop. There is no hardware, so arms are MuJoCo models, a production line is a loop, and every failure is one you inject yourself.

The manipulation does not have to work well. A scripted controller that lunges at roughly the right place is fine - none of the difficulty here is in making an arm grasp something. It is in the software around the arms.

The scenario

Arms work at stations on production lines across several sites. The line runs at a fixed beat, called a **takt**, one part arriving at each station every N seconds. Each cell repeats its cycle against that beat indefinitely. It does not wait for work and does not choose it.

- **Cells are not identical.** Different arm models, grippers, sensor sets, fixture geometry, and each with its own calibration that ages at its own rate.
- **A cell has to hold takt.** Succeeding slowly is its own failure, and a slow station becomes the constraint on the whole line.
- **Stations are coupled.** Upstream jams and a station is starved; downstream backs up and it is blocked. One cell stopping idles its neighbours.
- **The part changes.** On a mixed-model line the product variant switches from one part to the next, and each variant demands different reach, grasp, and sensors, and is supported by different policy versions.
- **Config comes from three places at once:** fleet defaults, site overrides, per-unit corrections.

What things are, in code

in the factory	in your build
a cell	a MuJoCo model plus a config record: hardware, sensors, calibration, takt
a part arriving	your loop releasing a cycle at the takt interval
the manipulation task	a scripted controller; quality is irrelevant
cycle succeeded or failed	a criterion you define
a camera or sensor	an entry in the cell's declared sensor set; the controller may read sim state directly
calibration	an offset you apply, plus an age; "expired" is an age check
a sensor dropping out	removing it from the declared set at runtime
the buffer between two stations	a queue of size N - empty means starved, full means blocked
a cell dying	however you choose to abort it mid-cycle
a variant change	swapping which task spec is in force

Phases

1: A design note, no code. Decide the structure before you build it.

- What the pieces are, what owns each one, and where the boundaries sit.
- The config precedence rule, and how a reader of a resolved config tells where a value came from.
- What a cell confirms before every cycle, not just at startup.
- What a cycle record must contain to be reproducible in six months.
- **The reasons a cell can fail to comply are not all the same.** Classify them, and say what each class should trigger and who needs to know.
- Your takt, your first two or three arm and gripper combinations, and what you are not building.

2: One cell running cycles, then a second with different hardware through the same code.

- At least two arm models with different actuator counts, and two grippers.
- Readiness that can fail mid-run: calibration expired, a needed sensor gone, the current variant out of reach or needing a grasp this cell lacks, takt no longer held.
- A refusal is a recorded outcome with a reason, not an exception that escapes.
- *Deliverable*: runnable code, a resolved config for one cell showing where each value came from, an example of each refusal.

3: Several stations running a shift, with faults, plus new hardware added by config.

- Parts on the beat, a variant changeover partway, and injected faults: a cell killed mid-cycle, calibration expiring under a running station, a sensor dropping out, one cell slowing past takt.
- The line keeps moving; the record accounts for work in flight and for the stations around the failure.
- Add a third arm model, a new gripper, and a new variant. Include the diff. Nothing in shared code should change.
- **A dashboard over the fleet**, reading the records your runtime writes. Per station: current state, cycles completed, refusals by reason, over-takt events, buffer levels either side. Plus a timeline across the shift showing each station running, blocked, starved, refused, or over takt. Being able to inject a fault and watch it propagate is the version worth demoing. Modest is fine — it is a view onto your records, not a product.
- *Deliverable*: the dashboard, the shift run behind it, and the extension diff.

4: One extension, OPTIONAL. Pick one:

- add ALOHA and report what assumed a single arm
- calibration lifecycle including what happens to cycles recorded against a calibration since superseded
- resume a shift killed halfway without redoing or losing work
- what a line supervisor needs on top of the dashboard to actually act

Models

`pip install mujoco robot_descriptions`. Verified to load (these are just examples, feel free to choose):

model	nq	nu
<code>panda_mj_description</code>	9	8
<code>ur5e_mj_description</code>	6	6

model	nq	nu
iiwa14_mj_description	7	7
robotiq_2f85_mj_description	8	1
aloha_mj_description (bimanual)	16	14

nq is position coordinates in the model state; nu is the number of actuators, i.e. the length of the control vector you write each step. First load of each clones the model repo, about a minute total - maybe do it early.

Language, stack and libraries are otherwise yours.

Submit

- Code, runnable, short README.
- **DESIGN.md** with your Phase 1 note, then per phase what you built and why. Include the config precedence rule, what each boundary owns, screenshots of the dashboard, the extension diff, what counts as a successful cycle and what that definition misses, and what you prioritized and why.